



*Software Requirements Specification
Version 1.0*

AssureX Claim Engine

Project Name: AssureX Claim Engine

Theme: AI-Powered Document Ops

Category: NextWave AI and ML

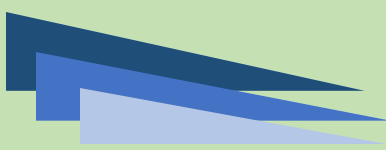


Table of Contents

1.1 Background and Necessity for the Application.....	2
1.2 Proposed Solution.....	3
1.3 Purpose of the Document	9
1.4 Scope of Project.....	9
1.5 Constraints.....	10
1.6 Functional Requirements.....	10
1.7 Non-Functional Requirements.....	18
1.8 Competition Integrity and Anti-Shortcut Requirements	19
1.9 Interface Requirements	20
1.9.1 Hardware	20
1.9.2 Software.....	20
1.10 Project Deliverables	22

1.1 Background and Necessity for the Application

Warranty claim management is vital for manufacturers and service centers, as it directly affects customer satisfaction, operational efficiency, and financial control. Traditional claim-evaluation methods require employees to manually examine purchase details, product age, fault descriptions, repair history, supporting documents, and warranty conditions. This process is time-consuming and prone to errors, inconsistent decisions, duplicate claims, overlooked exclusions, and delays in claim approval. The **AssureX Claim Engine** aims to automate and improve this process through Python-based Web development, machine learning, and intelligent claim-analysis techniques.

The '**AssureX Claim Engine**' reduces the requirement for completely manual warranty verification by automatically analyzing the information and documents submitted with a claim. The application analyses product details, purchase information, reported faults, repair history, warranty coverage, and supporting documents to determine whether a claim is valid, invalid, or requires manual review. It also generates a confidence score using a Python classification model and compares the result with a separately trained Google Teachable Machine model. By combining machine-learning predictions with warranty-condition checks, the application improves consistency, identifies uncertain or conflicting claims, and supports faster and more reliable decision-making. The goal is to deliver a secure, accurate, and accessible solution that enhances warranty claim processing for manufacturers and service centers.



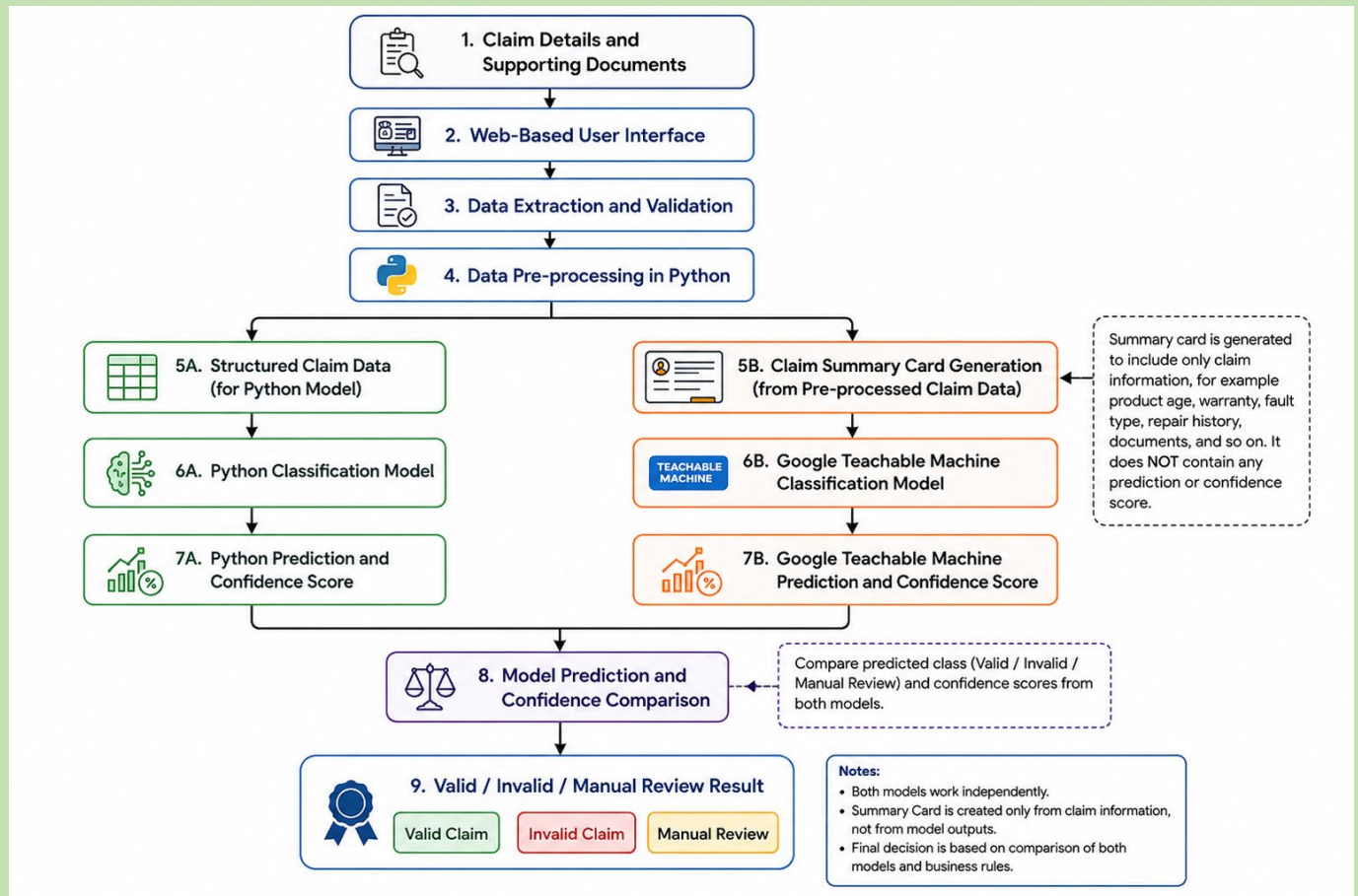
1.2 Proposed Solution

The proposed solution introduces an AI-powered, Web-based warranty claim validation application that helps manufacturers and service centers evaluate claims accurately and efficiently. Users enter product age, purchase information, fault details, repair history, warranty conditions, and supporting evidence through the application. The application processes these details using a Python-based classification model and predicts whether the claim is likely to be valid, invalid, or require manual review.

The **AssureX Claim Engine** generates confidence scores for all three claim categories and applies warranty validation rules to support the prediction. The same claim information is then converted into a visual Claim Summary Card and evaluated using a separately trained Google Teachable Machine model. The application compares the predicted class and confidence scores produced by both models. This approach improves reliability, identifies uncertain or conflicting claims, and supports faster and more consistent warranty decisions.



The sample architecture for the **AssureX Claim Engine** application is as follows:



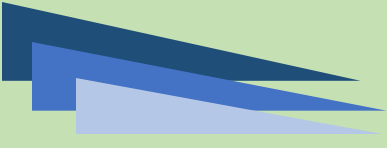
Sample Architecture of the Application

Reference Application: Students may refer to <https://iwarranty.ai/>

Students may refer to the iWarranty platform to understand the general design and workflow of an AI-enabled warranty and claims-management application. The reference platform may be used to understand concepts such as claim submission and intake, warranty-policy validation, claim triage, claim-status tracking, fraud or anomaly identification, claim adjudication, audit trails, compliance monitoring, and warranty analytics.

The reference application must not be copied or reproduced. Teams must design and develop their own original Python-based Web application and implement all functional and non-functional requirements specified in this SRS.

The competition application must independently process claim information using a **Python classification model** and a separately trained **Google Teachable Machine model**. It must generate prediction and confidence scores



from both models, compare their results, apply warranty-rule validation, and produce the final **Valid Claim, Invalid Claim, or Manual Review** decision.

The reference application is intended only to provide an understanding of warranty and claim-management workflows. The required Python classification, Google Teachable Machine comparison, confidence analysis, rule engine, and final three-class decision are specific requirements of this competition project and do not need to replicate the implementation used by the reference platform.

The development phase of the application includes following steps:

Step 1: Claim Data Collection

The user enters product details, purchase information, fault description, repair history, warranty conditions, and other required information through the Web-based interface. Supporting documents such as purchase receipts, warranty cards, product images, and repair records are also uploaded. Each submitted claim is assigned a unique Claim ID.

Step 2: Data Extraction and Validation

The application extracts relevant information from the uploaded documents and compares it with the details entered by the user. It checks missing fields, invalid dates, unsupported file formats, incorrect values, and inconsistencies such as mismatched serial numbers or purchase information.

Step 3: Data Pre-Processing

The validated claim data is cleaned and prepared for classification using Python. This includes handling missing values, converting date formats, encoding categorical fields, normalizing numerical values, and creating derived features such as product age, remaining warranty period, and number of missing documents.

Step 4: Dataset Creation

A structured warranty claim dataset is created containing Valid Claim, Invalid Claim, and Manual Review records. The dataset must include normal, incomplete, contradictory, complex, and borderline claim scenarios. The structured dataset is used to train and test the Python classification model.

Step 5: Python Classification Model Training

Different machine learning classification algorithms are trained and evaluated using Python. Their performance is compared using accuracy, precision, recall, F1-score, confusion matrix, and class-wise results. The best-performing model is selected and integrated into the Web application.

Step 6: Python Claim Classification

The pre-processed structured claim data is submitted to the trained Python model. The model predicts whether the claim is valid, invalid, or requires manual review and generates confidence scores for all three classes.

Step 7: Claim Summary Card Generation

The same pre-processed claim data is independently converted into a standardized visual Claim Summary Card. The card contains claim information such as product age, warranty period, fault category, repair history, receipt availability, serial-number status, and missing documents.

The Claim Summary Card must not contain the Python model prediction, confidence score, or final claim result.

Step 8: Google Teachable Machine Model Training

The Claim Summary Card images are grouped into three classes: Valid Claim, Invalid Claim, and Manual Review. These images are used to train a separate image-classification model using Google Teachable Machine.

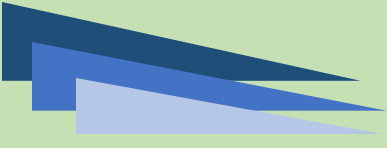


Step 9: Google Teachable Machine Classification

The Claim Summary Card generated for a submitted claim is provided to the trained Google Teachable Machine model. The model independently predicts the claim class and generates confidence scores for all three categories.

Step 10: Model Prediction and Confidence Comparison

The application compares the results produced by the Python classification model and the Google Teachable Machine model. It verifies whether both models predict the same class and calculates the difference between their confidence scores.



The confidence difference must be calculated using:

$$\text{Confidence Difference} =$$
$$| \text{Python Model Top-Class Confidence} - \text{Google Teachable Machine Top-Class Confidence} |$$

The two models are independently trained and therefore are not required to generate exactly identical confidence scores.

Step 11: Warranty Rule Validation

The application verifies business conditions such as warranty expiry, product age, fault coverage, purchase proof, serial-number match, previous repairs, excluded damage, duplicate claims, and missing documents. These checks support or challenge the predictions generated by the two models.

Step 12: Final Claim Decision

The final recommendation is generated by considering the Python model result, Google Teachable Machine result, confidence-score difference, warranty rules, missing evidence, and detected contradictions. The claim is then classified as Likely Valid, Likely Invalid, or Manual Review Required. Claims with low confidence, conflicting predictions, incomplete documents, or rule violations must be sent for manual review.

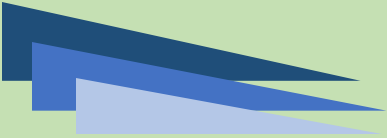
Hint: For this project, you are expected to create your own common warranty claim dataset containing a minimum of 1,500 unique claim records. No pre-existing dataset will be provided. The dataset must contain approximately equal records for the following classes:

- 500 Valid Claim records
- 500 Invalid Claim records
- 500 Manual Review records

The claim records must include different combinations of product age, purchase information, fault type, repair history, warranty conditions, serial-number status, missing documents, contradictions, and supporting evidence.

The same underlying claim records must be used for both models in different formats:

- For the **Python classification model**, the records must be stored as a structured CSV or tabular dataset.
- For the **Google Teachable Machine model**, the same records must be converted into visual Claim Summary Card images and placed under their corresponding classes.



The common dataset must be divided using a stratified split before generating the Claim Summary Cards:

- **70% Training:** 1,050 claim records
- **15% Validation:** 225 claim records
- **15% Testing:** 225 claim records

For Google Teachable Machine training, students must create at least **two visual variations of every training Claim Summary Card**, resulting in a minimum of **2,100 training images**. The variations may include changes in layout, font size, background, spacing, image quality, or date format without changing the claim information or class label.

Validation and testing Claim Summary Cards must not be uploaded for model training. They must remain separate for model evaluation and confidence-score comparison.

Each CSV record and its corresponding Claim Summary Card must have the same Claim ID, claim details, and class label. All visual variations of one claim must remain within the same dataset split. A claim used for validation or testing must not appear in the training data in either CSV or image form.



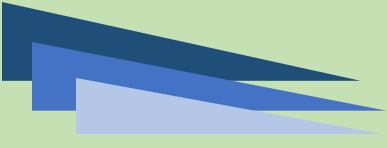
1.3 Purpose of the Document

The purpose of this document is to outline the design, functionality, and implementation plan of the proposed **AssureX Claim Engine**. This document ensures a shared understanding of the project objectives and provides a structured roadmap for the development, testing, and deployment of the application. It is intended for project stakeholders, developers, evaluators, manufacturers, and service-center representatives.

1.4 Scope of Project

The scope of this project is to design and implement an AI-powered, Web-based warranty claim validation application that assists manufacturers and service centers in evaluating warranty claims. The application will collect claim details and supporting documents, process the information using a Python classification model, compare the result with a Google Teachable Machine model, and classify the claim as Valid, Invalid, or Manual Review. It aims to reduce manual effort, improve decision consistency, identify incomplete or conflicting claims, and support faster claim processing. Integration with live manufacturer databases, payment systems, or enterprise warranty platforms falls outside the scope of this project.





1.5 Constraints

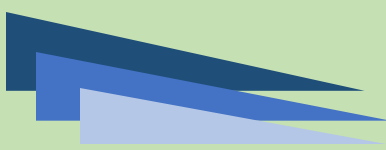
The **AssureX Claim Engine** project faces certain constraints such as dependency on the quality, completeness, and accuracy of the submitted claim details and supporting documents. The application's effectiveness is limited by the size, balance, and reliability of the training dataset. Differences may also occur between the confidence scores generated by the Python model and Google Teachable Machine. Privacy and security concerns regarding the storage and handling of customer, purchase, and warranty information must also be addressed in accordance with applicable data protection regulations.

1.6 Functional Requirements

Creating an **AssureX Claim Engine** involves several steps and components. The following are the functional requirements to develop such an application:

- i. **User Registration and Authentication** - The application should allow customers, service-center employees, claim reviewers, and administrators to register and securely log in. It should provide role-based access so that each user can access only the features and claim records permitted for their role.
- ii. **User Profile Management** - Users should be able to create and update their profiles containing basic contact information and account details. The application should maintain a unique User ID for each registered user.
- iii. **Product Registration** - The application should allow users to register products by entering details such as product name, category, brand, model number, serial number, purchase date, purchase price, retailer, and warranty duration. Each registered product should be assigned a unique Product ID.
- iv. **Warranty Record Management** - The application should store standard and extended warranty information, including warranty provider, start date, expiry date, coverage conditions, exclusions, and service-center details. Users should be able to view all active, expired, and approaching expiry warranties from a common interface.





- v. **Receipt and Invoice Upload** - Users should be able to upload and securely store purchase receipts, invoices, warranty cards, and other proof-of-purchase documents in formats such as PDF, JPG, JPEG, and PNG.



- vi. **Receipt Scanning and Data Extraction** - The application should extract important information from uploaded receipts and invoices, including purchase date, invoice number, product name, model number, serial number, retailer, purchase amount, and warranty duration.

- vii. **Extracted Data Verification** - The application should display the information extracted from uploaded documents and allow users to review and correct inaccurate or incomplete values before saving or submitting the claim.

- viii. **Warranty Tracking** - The application should automatically calculate warranty start and expiry dates and display the remaining warranty period. It should identify whether a product warranty is active, expired, nearing expiration, or covered under an extended warranty.

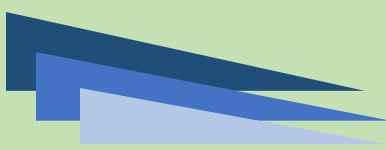
- ix. **Warranty Expiry Alerts** - The application should notify users when a registered product warranty is approaching its expiry date. Administrators should be able to configure the number of days before expiry when an alert should be generated.



- x. **Claim Registration** - The application should allow users or service-center employees to create a new warranty claim. Each submitted claim should be assigned a unique Claim ID and linked to the corresponding user, product, warranty, and supporting documents.

- xi. **Claim Information Collection** - The application should collect claim information such as product age, purchase details, fault occurrence date, fault description, damage type, warranty conditions, service history, previous replacement details, and claim submission date.

- xii. **Fault and Damage Evidence Upload** - Users should be able to upload product photographs, damage images, fault videos, serial-number photographs, diagnostic reports, and other supporting evidence required for claim evaluation.



- xiii. Repair History Management** - The application should record previous repair dates, repair-center details, replaced parts, repair outcomes, repair costs, and whether each repair was completed by an authorized or unauthorized service center.

- xiv. Document Organization** - The application should organize uploaded receipts, warranty cards, repair reports, product images, and other supporting documents under the relevant product and claim record. Users should be able to view, download, replace, or remove documents according to their access rights.



- xv. Data Validation** - The application should check mandatory fields, date formats, numerical values, uploaded file types, file sizes, and duplicate Claim IDs. It should prevent incomplete or invalid information from being submitted without appropriate warnings.

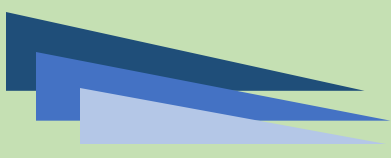
- xvi. Data Pre-Processing** - The submitted claim data should be cleaned and prepared using Python. This should include handling missing values, converting date formats, encoding categorical values, normalizing numerical values, and creating derived fields such as product age, remaining warranty period, and missing-document count.

- xvii. Common Warranty Claim Dataset** - The application should use a structured dataset containing valid, invalid, and manual-review warranty claim records. The same underlying claim records should be used in CSV or tabular form for the Python model and converted into Claim Summary Card images for Google Teachable Machine.



- xviii. Python Classification Model** - The application should use a machine learning classification model trained using Python. Students should train and compare suitable algorithms and select the best-performing model to classify a submitted claim as Valid Claim, Invalid Claim, or Manual Review.

- xix. Python Confidence Score Generation** - The Python classification model should generate confidence scores or class probabilities for all three claim categories. The application should display the predicted class and corresponding confidence values.



- xx. Claim Summary Card Generation** - The application should automatically convert the pre-processed claim information into a standardized visual Claim Summary Card. It should contain product age, warranty status, fault type, repair history, document availability, serial-number status, and other relevant claim information.



The Claim Summary Card should not contain the Python model prediction, Python confidence score, or final claim decision.

- xxi. Google Teachable Machine Classification** - The Claim Summary Card should be evaluated using a separately trained Google Teachable Machine image-classification model. The model should independently classify the claim as Valid Claim, Invalid Claim, or Manual Review and generate confidence scores for all three classes.

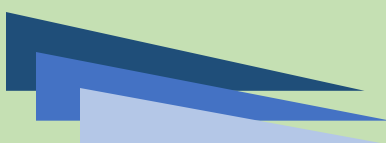
- xxii. Model Prediction Comparison** - The application should compare the class predicted by the Python classification model with the class predicted by the Google Teachable Machine model. It should clearly display whether the predicted classes match.

- xxiii. Confidence Score Comparison** - The application should compare the top-class confidence scores generated by the Python and Google Teachable Machine models. It should calculate and display the absolute difference between the two confidence scores.



- xxiv. Model Consistency Status** - The application should classify the comparison as Strong Match, Acceptable Match, Weak Match, Model Disagreement, or Uncertain Result based on configurable confidence-difference and minimum-confidence thresholds.

- xxv. Warranty Rule Validation** - The application should independently validate business rules such as warranty expiry, fault coverage, claim reporting period, proof of purchase, extended warranty, serial-number match, previous repairs, product replacement, excluded damage, and required documents.



- xxvi. Configurable Warranty Policies** - Warranty periods, covered faults, exclusions, reporting deadlines, mandatory documents, and repair conditions should be stored in configurable files or a database. The application should support different warranty rules for different product categories.



- xxvii. Serial-Number Verification** - The application should compare the serial number entered by the user with serial numbers extracted from the receipt, warranty card, product image, and repair records. A mismatch should generate a warning or route the claim for manual review.

- xxviii. Contraction Detection** - The application should identify conflicting information such as a claim date before the purchase date, a repair date before purchase, inconsistent product models, conflicting serial numbers, or a fault date after the claim submission date.

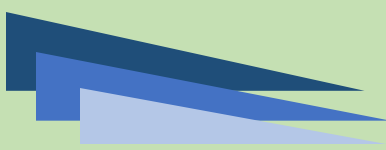
- xxix. Missing Document Detection** - The application should identify missing mandatory documents such as the purchase receipt, warranty card, product image, serial-number evidence, fault evidence, or repair report. It should inform the user which documents are required.

- xxx. Duplicate Claim Detection** - The application should detect possible duplicate claims by comparing Claim IDs, invoice numbers, product serial numbers, fault descriptions, claimant details, document hashes, and previous claim records.



- xxxi. Document Duplicate Detection** - The application should create a secure hash for uploaded documents and detect whether the same receipt, invoice, warranty card, or evidence file has already been used in another claim.

- xxxii. AI-Generated Claim Summary** - The application should generate a clear summary of the product, warranty coverage, reported fault, repair history, uploaded evidence, detected issues, and claim status to help users and reviewers understand the claim quickly.



- xxxiii. Claim Preparation Assistance** - Before final submission, the application should guide the user by displaying missing information, missing documents, approaching deadlines, possible contradictions, and recommended corrective actions.



- xxxiv. Final Claim Decision** - The application should generate the final result by considering the Python model prediction, Google Teachable Machine prediction, confidence-score difference, warranty rules, missing documents, duplicate indicators, and contradictions.

The final result should be one of the following:

- Likely Valid
- Likely Invalid
- Manual Review Required

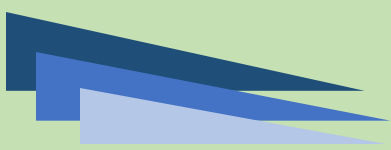
- xxxv. Decision Explanation** - The application should explain the factors supporting the decision, factors opposing the decision, rules passed, rules failed, detected contradictions, and additional evidence required.

- xxxvi. Manual Review Workflow** - Claims with low confidence, conflicting model predictions, missing evidence, duplicate indicators, or rule violations should be added to a manual-review queue. An authorized reviewer should be able to approve, reject, or request additional information.



- xxxvii. Reviewer Comments and Decision Override** - Authorized reviewers should be able to add comments and override the automated recommendation. The original AI results and reasons for the override should remain available in the audit history.

- xxxviii. Claim Status Tracking** - Users should be able to track the progress of a submitted claim through stages such as Draft, Submitted, Under Evaluation, Additional Information Required, Manual Review, Approved, Rejected, and Closed.

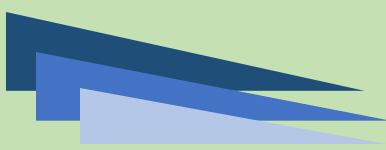


- xxxix. Notification and Alerts** - The application should notify users about warranty expiry, claim submission, missing documents, requests for additional information, status changes, review completion, approval, or rejection.



- xl. Claim Dashboard** - Users should have access to a dashboard displaying registered products, active warranties, expiring warranties, saved receipts, submitted claims, pending actions, and recent claim decisions.
- xli. Administrator Dashboard** - The application should provide administrators with a dashboard displaying total claims, valid claims, invalid claims, manual-review claims, pending claims, duplicate alerts, model disagreements, average confidence scores, and claim trends.
- xlii. Search and Filtering** - Users and administrators should be able to search and filter warranty and claim records using Claim ID, Product ID, product category, serial number, warranty status, claim status, risk level, confidence range, reviewer, and date range.
- xliii. Data Analysis and Reporting** - The application should generate analytics on claim outcomes, frequently reported faults, rejected claim reasons, product categories, warranty expirations, repair patterns, model performance, and manual-review frequency.
- xliv. Downloadable Claim Report** - The application should generate a downloadable report containing the claim details, uploaded evidence, warranty status, Python model result, Google Teachable Machine result, confidence comparison, rule-validation result, contradictions, final recommendation, and reviewer comments.
- xlv. Data Export** - Administrators should be able to export selected claims, product, warranty, and analytics records in CSV or Excel-compatible format.





xlvi. Data Storage - The application should use a secure relational or NoSQL database to store user accounts, products, warranties, receipts, claims, repair histories, predictions, confidence scores, rule results, notifications, and audit records.

xlvii. Audit Trail - The application should record important actions such as account creation, product registration, document upload, extracted-data correction, model prediction, claim submission, status change, reviewer action, and final decision.

xlviii. Model Version Tracking - Each claim prediction should be linked to the version of the Python and Google Teachable Machine models used during evaluation. Updating a model should not alter previously recorded results.

xl ix. Error Handling - The application should display understandable error messages for invalid files, incomplete claim data, failed document extraction, unavailable models, database errors, and unsuccessful predictions without exposing technical system details.



1. Monitoring and Anomaly Alerts - The application should monitor failed uploads, repeated login attempts, duplicate documents, unusual claim activity, model failures, low-confidence predictions, and excessive disagreement between the two models. Appropriate alerts should be generated for administrators.

In addition, the application should provide a responsive and user-friendly Web interface that can be accessed through desktop, tablet, or mobile browsers. The interface should allow users to organize receipts, track warranties, prepare claims, view AI-generated summaries, monitor claim progress, and receive alerts before warranty or claim deadlines. These capabilities reflect the workflow demonstrated by the reference application while extending it with claim validation, dual-model comparison, business-rule verification, duplicate detection, and administrative review.

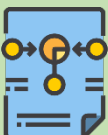


1.7 Non-Functional Requirements

There are several non-functional requirements that should be fulfilled by the application which are as follows:



1. **Performance:** The application should process uploaded claim details and generate predictions from both models within five seconds under normal operating conditions.
2. **Scalable:** The application should be able to store and manage at least 10,000 warranty claims and support multiple manufacturers, service centers, products, and concurrent users without significant performance degradation.
3. **Usable:** The application should provide an intuitive and user-friendly Web interface for customers, service-center employees, reviewers, and administrators.



4. **Accuracy:** The Python classification model and Google Teachable Machine model should achieve an overall accuracy of at least 85% on unseen test claims and should reliably identify claims requiring manual review.

5. **Available:** The application should remain operational during business hours with at least 99% up time to ensure uninterrupted claim submission, evaluation, tracking, and review.



These are the bare minimum expectations from the project. It is a must to implement the **FUNCTIONAL** and **NON-FUNCTIONAL** requirements given in this SRS.

Once they are complete, you can use your own creativity and imagination to add more features if required.

1.8 Competition Integrity and Anti-Shortcut Requirements

To ensure that the project represents genuine team effort, the following requirements are mandatory:

1. Each team member must explain the modules assigned to them.
2. The GitHub repository must contain meaningful commits across all five competition days.
3. Teams must maintain a development log showing:
 - Work completed
 - Problems encountered
 - Model failures
 - Changes made
 - Tests performed
4. Evaluators may select any function and ask the team to explain it.
5. Teams must complete a surprise modification during final evaluation, such as:
 - Adding a new warranty exclusion
 - Changing a confidence threshold
 - Supporting another date format
 - Adding a new contradiction rule
 - Modifying the decision logic
 - Adding a new dashboard filter
6. Teams may be required to fix a deliberately introduced defect.
7. Unused code, unexplained code, hidden external decision APIs, or hard-coded test answers may result in disqualification.
8. The application will be evaluated using hidden claims unavailable before the final assessment.
9. Any AI tool used during development must be declared in an AI_USAGE.md file, including:

- Tool name
 - Purpose of use
 - Modules affected
 - Changes made by the students
 - Testing performed by the students
10. AI-generated output cannot replace technical understanding. Failure to explain or modify submitted code may result in the affected module receiving zero marks.



1.9 Interface Requirements

1.9.1 Hardware

Intel Core i5/i7 Processor or higher
 8/16 GB RAM or higher
 Color SVGA monitor
 500 GB Hard Disk space
 Mouse and Keyboard

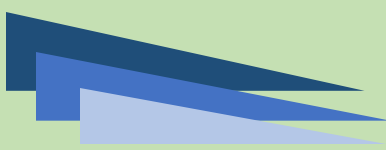
1.9.2 Software

Technologies to be used:

1. **Frontend:** HTML5, CSS3, JavaScript, Bootstrap, Streamlit, React, or any other suitable frontend technology
2. **Backend:** Flask, Django, or FastAPI

3. **Assets and Data Store:** CSV files, JSON files/YAML, PDF documents and JPG, JPEG, or PNG images
4. **Database:** Firebase, MongoDB, MySQL, PostgreSQL, SQLite, or any other relational or NoSQL database
5. **Programming/IDE:** Python, Jupyter Notebook, Anaconda, Google Colab, PyCharm, or Visual Studio Code
6. **Machine Learning:** Scikit-learn, XGBoost, TensorFlow, Keras, PyTorch, and Google Teachable Machine
7. **Libraries:** NumPy, Pandas, Matplotlib, Seaborn, OpenCV, Pillow, Joblib, Pickle, and OCR libraries such as Tesseract or EasyOCR
8. **Version Control and Deployment:** Git, GitHub, Render, Railway, PythonAnywhere, or Streamlit Community Cloud





1.10 Project Deliverables

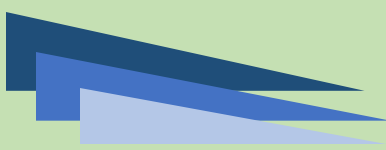
Each team must design, build, test, document, deploy, and demonstrate the complete **AssureX Claim Engine** application.

The final submission must contain the following items:

1. Project Report

The project report must include:

- Problem definition
- Background and business necessity
- Proposed solution
- Purpose of the document
- Scope of the project
- Assumptions
- Constraints
- Functional requirements
- Non-functional requirements
- Application architecture
- Module descriptions
- Database design
- Data dictionary
- Data Flow Diagram
- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Decision Flow Diagram
- Rule-engine design
- Python classification model design
- Google Teachable Machine model design
- Dataset description



- Dataset-generation method
- Data preprocessing and feature-engineering steps
- Python model-training procedure
- Google Teachable Machine training procedure
- Model evaluation
- Confusion matrix
- Accuracy, precision, recall, and F1-score
- Model prediction and confidence comparison
- Testing strategy
- Security considerations
- Privacy considerations
- Limitations
- Future enhancements

2. Source Code

The complete source code must be uploaded to a public GitHub repository.

The repository should include the following files and folders:

README.md

AI_USAGE.md

requirements.txt

LICENSE

src/

templates/

static/

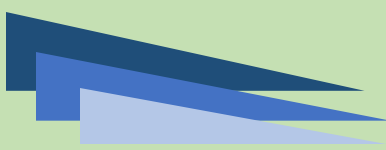
data/

notebooks/

model/

policies/

dataset_generator/



database/
tests/
sample_claims/
documentation/
screenshots/
reports/
config/

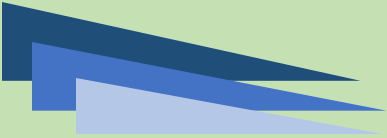
The source-code submission must include:

- Python frontend and backend files
- HTML, CSS, JavaScript, or Streamlit files
- Database scripts
- Dataset-generation scripts
- Data-preprocessing scripts
- Python model-training scripts
- Saved Python model files
- Label encoders and preprocessing files
- Claim Summary Card generation scripts
- Google Teachable Machine exported model files
- Warranty-policy files
- Rule-engine files
- OCR and document-processing files
- Automated test scripts
- Sample claim data

3. Dataset

The submission must include one common warranty claim dataset represented in two formats.

For the Python classification model, the dataset must be available as **structured CSV or tabular records**.



For **Google Teachable Machine**, the same underlying claim records must be converted into **Claim Summary Card images**.

The dataset submission must include:

- Structured CSV training dataset
- Structured CSV validation dataset
- Structured CSV testing dataset
- Claim Summary Card training images
- Claim Summary Card validation images
- Claim Summary Card testing images
- Dataset-generation scripts
- Claim scenario definitions
- Dataset labels
- Dataset statistics
- Data dictionary
- Claim ID and image filename mapping file

Each CSV record and its corresponding Claim Summary Card must contain the same Claim ID, claim details, and class label.

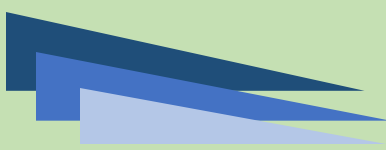
The team must clearly separate the training, validation, and testing data. A claim used for validation or testing must not appear in the training dataset in either CSV or image form.

4. Python Classification Model Evidence

Teams must submit complete evidence of the Python classification model.

The submission must include:

- Structured dataset used for model training
- Data-preprocessing notebook or script
- Feature descriptions
- Feature-engineering steps
- Missing-value handling method
- Categorical encoding method



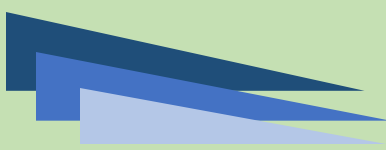
- Numerical scaling or normalization method
- Algorithms tested
- Hyperparameter settings
- Training results
- Validation results
- Testing results
- Cross-validation results
- Confusion matrix
- Accuracy
- Precision
- Recall
- F1-score
- Class-wise performance
- Feature-importance analysis, where applicable
- Saved Python model file
- Label encoder files
- Preprocessing pipeline files
- Model version
- Sample test predictions

Students must train and compare **at least three suitable machine learning classification algorithms before selecting the final Python model.**

5. Google Teachable Machine Evidence

Teams must submit:

- Google Teachable Machine project link or permitted project evidence
- Screenshots of the three classes
- Valid Claim class samples
- Invalid Claim class samples
- Manual Review class samples



- Training configuration
- Number of images per class
- Training observations
- Incorrectly classified samples
- Retraining details
- Exported Google Teachable Machine model
- Label file
- Model version
- Model test screenshots
- Sample Claim Summary Cards

The Claim Summary Cards used for Google Teachable Machine must not contain the Python prediction, Python confidence score, or final claim decision.

6. Model Prediction and Confidence Comparison Report

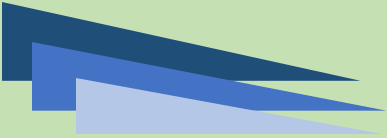
A separate model-comparison report must contain results for at least 30 unseen test claims.

Each test claim must include:

- One structured claim record for the Python model
- One corresponding Claim Summary Card for Google Teachable Machine

The report must include:

- Claim ID
- Actual claim class
- Python model predicted class
- Python confidence for Valid Claim
- Python confidence for Invalid Claim
- Python confidence for Manual Review
- Claim Summary Card filename
- Google Teachable Machine predicted class
- Google Teachable Machine confidence for Valid Claim

- 
- Google Teachable Machine confidence for Invalid Claim
 - Google Teachable Machine confidence for Manual Review
 - Predicted-class match status
 - Top-class confidence difference
 - Model consistency status
 - Warranty-rule result
 - Missing documents
 - Contradictions detected
 - Duplicate-claim indicators
 - Final application decision
 - Explanation of major disagreements
 - Overall comparison summary

The application should classify the comparison as:

- Strong Match
- Acceptable Match
- Weak Match
- Model Disagreement
- Uncertain Result

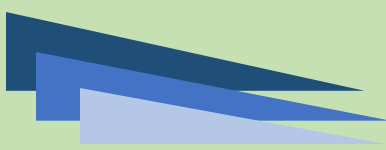
Claims with different model predictions, low confidence, missing evidence, or large confidence differences must be sent for manual review.

7. Warranty Policy Files

At least three configurable warranty-policy files must be submitted.

Each policy must include:

- Product category
- Coverage duration
- Warranty start conditions
- Covered faults
- Exclusions



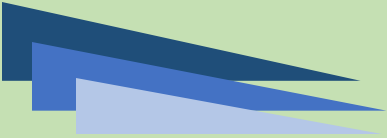
- Claim-reporting period
- Repair conditions
- Authorized service-centre requirements
- Replacement conditions
- Grace periods
- Mandatory documents
- Hard-fail rules
- Warning rules
- Manual-review rules

The warranty rules must be stored in JSON, YAML, CSV, or database format. They must not be hard-coded throughout the application.

8. Test Cases

The submission must include:

- Functional test cases
- Integration test cases
- Boundary test cases
- Negative test cases
- Security test cases
- Database test cases
- OCR test cases
- Python model test cases
- Google Teachable Machine test cases
- Model-comparison test cases
- Rule-engine test cases
- Contradiction-detection tests
- Missing-document tests
- Duplicate-claim tests
- Serial-number mismatch tests

- 
- Low-confidence test cases
 - Model-disagreement test cases
 - Hidden-test readiness checklist

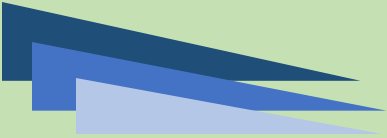
The project must demonstrate at least:

- One valid claim
- One invalid claim
- One manual-review claim
- One expired-warranty claim
- One missing-document claim
- One duplicate claim
- One contradictory claim
- One serial-number mismatch
- One unauthorized-repair claim
- One tricky boundary-date claim
- One case where the two models disagree

9. Installation Instructions

The documentation must provide:

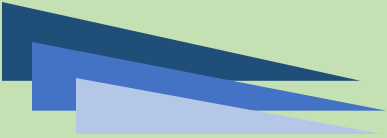
- Prerequisite software
- Supported operating system
- Required Python version
- Virtual-environment creation steps
- Dependency installation steps
- Database configuration
- Database initialization or seed steps
- Model placement instructions
- Google Teachable Machine model placement
- OCR installation and configuration
- Environment-variable configuration

- 
- Folder setup
 - Application execution command
 - Test execution command
 - Default administrator setup
 - Troubleshooting instructions

10. Execution Instructions

The **README.md** file must explain:

- How to install the application
- How to register or log in
- How to register a product
- How to add warranty information
- How to create a claim
- How to upload documents
- How to verify extracted information
- How to submit a claim
- How to generate the Python prediction
- How to interpret Python confidence scores
- How to generate the Claim Summary Card
- How to obtain the Google Teachable Machine prediction
- How to compare both model results
- How to review warranty-rule results
- How to check contradictions
- How to identify duplicate claims
- How to access the manual-review queue
- How to access the administrator dashboard
- How to track claim status
- How to export a claim report
- How to run automated tests



11. GitHub Repository

The GitHub repository must:

- Have public access
- Contain meaningful commit history
- Include commits from all team members
- Include complete source code
- Include model and configuration instructions
- Include the structured dataset or dataset-generation instructions
- Include Claim Summary Card samples
- Avoid confidential or real customer information
- Include known limitations
- Include assumptions
- Include sample evaluator login details
- Include database initialization instructions
- Include screenshots of the working application
- Include test results
- Include the project report or report link
- Include the blog link
- Include the demonstration video link

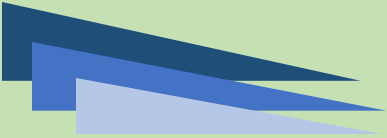
The public GitHub URL must be submitted with the project.

12. Deployed Application

Teams should submit a publicly accessible deployment URL.

The deployed application submission should include:

- Public application URL
- Evaluator login credentials
- Administrator login credentials
- Sample claim records
- Instructions for testing the application



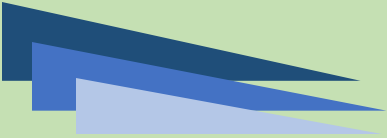
Where public deployment is not possible, teams must provide complete local installation instructions and a working application demonstration.

13. Demonstration Video

A demonstration video in .mp4 format is mandatory.

The video must demonstrate:

- User registration or login
- Product registration
- Warranty registration
- Claim creation
- Receipt and document upload
- OCR extraction
- Extracted-data verification
- Data preprocessing
- Python classification model prediction
- Python confidence scores for all three classes
- Claim Summary Card generation
- Google Teachable Machine classification
- Google Teachable Machine confidence scores
- Comparison of both model predictions
- Confidence-difference calculation
- Warranty-rule execution
- Missing-document detection
- Contradiction detection
- Duplicate-claim detection
- Manual-review routing
- Reviewer comments or override
- Administrator dashboard
- Claim status tracking

- 
- Report generation
 - One valid claim
 - One invalid claim
 - One manual-review claim
 - One tricky boundary case
 - One model-disagreement case

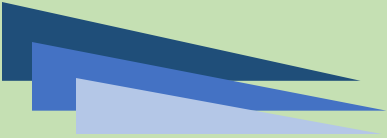
Submission of the demonstration video is mandatory.

14. Technical Blog

Each team must publish a technical blog of at least 2,000 words on Blogger, Tumblr, Ghost, Medium, or another free blogging platform.

The blog must discuss:

- Business problem
- Background and necessity
- Proposed solution
- Application architecture
- Dataset creation
- Dataset challenges
- Python model development
- Algorithms compared
- Google Teachable Machine training
- Claim Summary Card generation
- Python integration
- Model prediction comparison
- Confidence-score comparison
- Warranty-rule design
- OCR and document processing
- Difficulties encountered
- Model errors

- 
- Model disagreement cases
 - Testing results
 - Security considerations
 - Limitations
 - Lessons learned
 - Future enhancements

The published blog link must be included in the project documentation and GitHub repository.

15. AI Tool Usage Declaration

Teams must not directly submit content or code generated by GPTs or other AI tools without independently reviewing, modifying, testing, and understanding it.

Any AI tool used during development must be declared in the ``AI_USAGE.md`` file.

The declaration must include:

- Name of the AI tool
- Purpose of use
- Prompt or type of assistance requested
- Files or modules affected
- Modifications performed by the team
- Testing completed by the team
- Name of the team member who verified the output

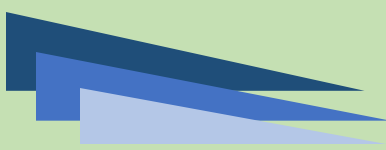
AI-generated images may be used for visual representation, but the tool used must be declared.

The final claim decision must not be generated through an external generative-AI API. It must be produced using the team's Python classification model, Google Teachable Machine model, warranty-rule engine, and application logic.

16. Final Submission Checklist

The final submission must include:

- Project report
- Public GitHub repository URL



- Complete source code
- Structured warranty claim dataset
- Claim Summary Card image dataset
- Python model files
- Google Teachable Machine model files
- Warranty-policy files
- Model-comparison report
- Test cases and results
- Installation instructions
- Execution instructions
- Deployment URL, where applicable
- Demonstration video
- Technical blog link
- AI_USAGE.md declaration
- Team contribution record

Documentation is a very important part of the project. All important aspects of the application, dataset, models, rules, testing, installation, execution, security, and limitations must be documented completely and clearly.

~~~ End of Document ~~~